```
}

# Bind the ruleset to tcp input
input(
      type="imtcp"
      port="514"
      ruleset="rsRemoteTCP"
)
```

This host has to:

1. Receive all the log messages from Ubuntu1404lts1, and…
2. Send them to the local MySQL database that we have previously created.

Note: A Ruleset can consist of multiple rules and each rule can have multiple action objects. In our example, Ruleset *rsRemoteTCP* has only one rule *.*, and it has only one *action(…)* object to store the message in the database. Multiple actions should be combined with the '&' symbol in between. Most of the attributes of *action(…)* and *input(…)* are self-explanatory, so we can skip that discussion.

Now restart the Rsyslog service on both the hosts as follows:

```
sudo service rsyslog restart
```

To test the set-up, let's write a small C program called *test_logger.c* on Ubuntu1404lts1 as shown below:

```
#include <syslog.h>
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char* argv[])
{
      unsigned int count, index;
      count=( argc==2 )? atoi(argv[1]) : 10;
      openlog("TestLogger",LOG_PID,LOG_LOCAL7);
      for(index=0;index<count;index++)
      {
        printf("Test log generated [count=%d]\n",index);
        syslog(LOG_INFO,"Test log message %d",index);
        sleep(1);
      }

      closelog();

      return 0;
}
```

Compile the program as follows:

```
gcc test_logger.c -o test_logger
```

Run the executable on Ubuntu1404lts1 with the number of logs to be generated as the argument, by using the following command:

```
./test_logger 5
```

To check for log messages in the database, log into the MySQL server on Ubuntu1404lts2 and run the SQL query as follows:

```
mysql -u rsysloguser -p
<<Enter password>>
mysql>USE Syslog;
mysql>SELECT ID,fromHost,Message
     ->FROM SystemEvents
     ->WHERE Message LIKE '%Test log%';
+------+--------------+--------------------+
| ID   | fromHost     | Message            |
+------+--------------+--------------------+
| 1015 | ubuntu1404lts1 |  Test log message 0 |
| 1016 | ubuntu1404lts1 |  Test log message 1 |
| 1017 | ubuntu1404lts1 |  Test log message 2 |
| 1018 | ubuntu1404lts1 |  Test log message 3 |
| 1019 | ubuntu1404lts1 |  Test log message 4 |
+------+--------------+--------------------+
5 rows in set (0.00 sec)
mysql>
```

So, we can see that all our test messages from Ubuntu1404lts1 are successfully logged in the database.

A lot more customisation can be done with Rsyslog configuration. You can create your own message format, filter the messages based on facility, priority, source or other criteria. Message transmission can be made secure using TLS (SSL). Apart from computers, now-a-days, network elements also come with Rsyslog support, using which you can receive log messages directly from network switches and routers in such a centralised logging set-up. So, interested readers should definitely refer to the official Rsyslog manual and explore all other features and possibilities. END

References

[1] *http://www.rsyslog.com*
[2] *http://www.tutorialspoint.com/unix/unix-system-logging.htm*
[3] *http://tecadmin.net/setup-centralized-logging-server-using-rsyslogd/*
[4] *http://www.techrepublic.com/blog/linux-and-open-source/set-up-rsyslog-to-store-syslog-messages-in-mysql/*
[5] *http://www.rsyslog.com/doc/v8-stable/tutorials/tls_cert_summary.html*

By: Ramaprasad Chakraborty

The author is a scientist engineer SD at Sriharikota, Indian Space Research Organisation (ISRO) and has six years of industry experience. He can be reached at *chakrabortyramaprasad@gmail.com*